
Competence is not Performance: Where Coding Agents Actually Parse Along the Language-Complexity Hierarchy

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Agentic systems for the molecular sciences rest on an unglamorous first step: turn-
2 ing a simulation’s raw output into structured data by **parsing**. This is assumed to
3 work because a coding agent emits Turing-complete code—a claim about expres-
4 sive *competence*, not realized *performance* (Chomsky’s distinction). We make it
5 measurable: on real computational-chemistry output we vary an MCP parser har-
6 ness from no support to complexity-aware, running a first sweep (159 runs, Claude
7 Opus 4.8) that scores success, workload, and failure modes by grammatical class.
8 The result is not the obvious one. Merely exposing tested parsers barely changes
9 success, raises workload, and *introduces* silent interface failures the from-scratch
10 baseline never produces; what removes them—at near-baseline cost—is telling
11 the agent where the source sits on the language hierarchy and where each tool’s
12 power ends, while reasoning failures persist. Realized parsing performance thus
13 depends less on *access* to tested tools than on *knowing* their power relative to the
14 task.

15 1 Introduction

16 When a coding agent is deployed in a scientific workflow, it is commonly assumed to handle what-
17 ever data processing the workflow requires. The underlying argument is that the agent writes code,
18 that code is a Turing-complete medium, and that such a medium can express any computable trans-
19 formation; any computable task should therefore be within reach. The assumption underlies the use
20 of agents as general-purpose connectors between scientific tools and the data they emit.

21 This conflates two notions Chomsky long ago separated: *competence*, what a system can in prin-
22 ciple represent, and *performance*, what it reliably does [Chomsky, 1965]. The “agents are Turing-
23 complete” claim is entirely about the first. That an agent can *emit* a Turing-complete language is
24 trivially true; and the stronger expressivity results—that a transformer’s forward pass is Turing-
25 complete [Pérez et al., 2021], or that chain-of-thought extends its reach [Merrill and Sabharwal,
26 2024]—hold only under idealized unbounded precision and unbounded computation. Performance
27 is a separate, empirical question, and where it has been measured the two diverge: networks with
28 ample capacity to fit their training data still fail to generalize across the Chomsky hierarchy [Delé-
29 tang et al., 2023], and frontier models that could in principle decompose a compositional problem
30 instead rely on shortcut pattern-matching and degrade as it grows [Dziri et al., 2023]. We therefore
31 treat realized capability as a hypothesis to be measured, not inferred from expressive power.

32 Measuring it requires a principled notion of what makes one processing task harder than another, and
33 formal language theory provides one. The Chomsky hierarchy [Chomsky, 1956] ranks a language
34 by the machinery its recognizer needs. A *regular* language can be read with finite state alone—a

regular expression; a *context-free* language needs a stack, to follow nesting and recursion; a *context-sensitive* language must enforce agreement or counting across the whole input, as in “declare N items, then produce exactly N ,” which no stack alone can do; and at the top, *recursively enumerable* languages allow arbitrary and possibly non-terminating computation.

Because the jump from context-free to context-sensitive is so wide, a finer band was singled out between them—Joshi’s *mildly context-sensitive* languages [Joshi, 1985] and the LCFRS fan-out ladder [Seki et al., 1991]—which captures bounded counting and cross-serial dependencies while still parsing in polynomial time. It is also the band into which most of the real scientific output we study falls (§4.1).

Grammatical class is only half of what makes a format hard in practice. The other half is whether a grammar is actually *available*—whether someone has already designed and tested a parser for it—and this second factor can run counter to the first: a context-free XML file is grammatically more complex than a regular, ad-hoc log, yet far easier to extract, because its grammar has been supplied. Separating intrinsic class from grammar-availability allows a success or failure to be attributed to the correct cause; the harness acts on the second factor, not the first.

Parsing is a well-suited probe for this gap. It is the first step of nearly every information-processing chain and among the least examined, yet on real inputs it already spans the whole hierarchy, from trivially regular tables to cross-referential multi-file formats. The nearest prior measurements grade language models on formal-language recognition across the Chomsky hierarchy as input length grows [Dong et al., 2026, Petty et al., 2025]. These are the closest prior work, but they measure a different task: they use synthetic strings and ask the model to *recognize* membership, whereas we use real, heterogeneous scientific output and ask the agent to *build a working artifact*—a parser or extraction pipeline that returns correct structured data. And instead of a single accuracy number, we vary the support the agent is given and index its failures by grammatical class, so as to identify not only where performance degrades but the associated failure modes and the conditions that recover it.

The output we parse is real computational-chemistry data: the logs and restart files that ab initio simulations produce. They are a demanding instance: heterogeneous across codes, weakly standardized, and dense with the counting and cross-reference structure that occupies the middle of the hierarchy. A silent parsing failure here becomes a silent *scientific* failure, as the downstream analysis is then grounded on no data.

2 The system under test: an MCP parser harness

By a *parser harness* we mean a layer that wraps a collection of independently developed parsers behind one uniform interface—with documentation and machine-readable metadata—so that a caller can discover which parser handles a given format, select it, and invoke it without re-implementing the parsing itself. The instrument for this study is one such harness: a dual-mode scientific parsing framework built on the Model Context Protocol (MCP) [Anthropic, 2024], first released as an open-source project. It wraps a set of existing, independently maintained community parsers (plus, for this study, a coarse regex parser standing in for regex-class tools) and exposes each of them in two ways: as an MCP tool that an agent can discover and call interactively, and as a direct Python import that a production pipeline can depend on. Both exposures return the same Pydantic-validated output model, so the data contract is identical whether a parser is called as a tool or imported. Around the parsers sits a documentation layer—per-parser descriptions of the formats each one handles, the schema of that output model, machine-readable capability metadata, and prompt scaffolding.

The harness was conceived to serve parser *onboarding* into a FAIR materials repository, which ingests the outputs of dozens of simulation codes and relies on a hand-written, tested parser for each one. Onboarding a new code is therefore gated on someone authoring and validating that parser—a slow, manual, and easily-stale bottleneck. The harness targets it directly: it automates the testing and deployment of parsers as they are contributed, so a new parser is checked against real files and made discoverable rather than re-implemented. Its broader aim is to turn parsers into shared, tested, discoverable community components, so that parsing capability—and with it the machine-readability of scientific output—is exchanged across the community rather than rebuilt in isolation each time [Ghiringhelli et al., 2023]. Exposing those same components to coding agents, the setting we study here, is a natural extension of that goal.

89 This layered design also makes the framework a clean measurement instrument. Because the parsers,
 90 their documentation, and their tool exposure are separable, we can give the agent any subset of them
 91 while holding everything else fixed—varying how much the harness provides and measuring the
 92 effect on what the agent produces.

93 3 Method

94 The experiment varies difficulty along two axes. The first is the *source*: the grammatical class of
 95 the format being read—regular, context-free, mildly context-sensitive, or context-sensitive—held
 96 apart from grammar-availability, so that having or lacking a tested parser is measured as its own
 97 factor rather than folded into the class. The second is the *target*: how hard the artifact the agent must
 98 *produce* is. This is a generation problem, not a recognition one, and is therefore not a second instance
 99 of the Chomsky hierarchy: the output’s surface syntax (a JSON object, a Python script) is cheap to
 100 emit and, if re-parsed downstream, cheap to recognize. What scales here is *semantic* correctness:
 101 a bounded target (fill a fixed schema) has a finite, field-checkable output space, whereas a Turing-
 102 complete target (generate an executable artifact such as a PySCF input script) has an unbounded one
 103 whose correctness is undecidable in general. Source difficulty is thus a question of *recognition* (the
 104 Chomsky hierarchy), target difficulty a question of *synthesis* and verifiability—two distinct axes.

105 We vary how much of the harness the agent receives. In every arm it is given the source file, the
 106 target schema, and the ability to run code; the arms differ only in the support layered on top. At
 107 one extreme (NONE) the agent gets no harness at all and must write its parser from scratch. From
 108 there we add support one layer at a time: exposing the parsers as callable tools but nothing more
 109 (BARE), then adding their per-parser documentation (DOCS), then a cross-parser capability map of
 110 which tool recovers which property (FULL), and finally an annotation of the source’s grammatical
 111 class together with each tool’s parsing-power ceiling (COMPLEXITY). The step from NONE to the
 112 rest isolates the value of the harness itself; the steps among the remaining arms isolate the value of
 113 the documentation built on top of it.

114 All runs use a single coding agent (Claude Opus 4.8) driven through the Claude Agent SDK, with
 115 identical task prompts and per-run turn and budget caps; only the arm’s system-prompt support
 116 differs. Per run we record tool calls, turns, tokens, the agent’s *chain-of-thought*, and a failure-
 117 mode taxonomy scored by conservative, transcript-grounded detectors (wrong-parser, silent-empty-
 118 accepted, parser-from-scratch, unit-error, retry-loop, gave-up), indexed by grammatical class $\times$ arm.
 119 Correctness is scored field-level against a reference built from the union of succeeding parsers.

120 4 Results

121 4.1 Preliminary: the grounding layer and the grammar-availability effect

122 Our test corpus is 1,104 real output files spanning 16 simulation codes, drawn from a public
 123 computational-materials database [Draxl and Scheffler, 2019] whose persistent identifiers let each
 124 file be traced. By grammatical class it is heavily skewed toward the middle of the hierarchy: of the
 125 primary-output files, 44 of 51 are mildly context-sensitive free-form logs and only four are context-
 126 free formats with a supplied grammar. The two extremes below Turing-completeness are barely
 127 present—none is purely *regular* (Chomsky type 3, finite-state) and only three are *context-sensitive*
 128 (type 1, cross-referential multi-file), too few to test. Our measurements therefore speak to the middle
 129 two rungs, which are also the common case for real ab initio output; we return to the extremes in the
 130 outlook.

131 As a component measurement motivating the axes, we ran the four wrapped parsers themselves—
 132 three maintained community libraries plus a coarse regex parser standing in for regex-class tools—
 133 directly on these files, scoring whether an atomic structure was recovered (Table 3). Each parser
 134 targets only a few of the codes, so no single one recovers structure from more than a quarter of the
 135 primary-output files—low by specialization, not by defect. Even a best-of-four selector that accepts
 136 *any* parser’s output, i.e. an upper bound on what routing among them could achieve, recovers only
 137 45%: the available parser set collectively covers under half of this heterogeneous space. Its failures
 138 are often *silent*—the parser returns grammatically well-formed output that is nonetheless void or
 139 semantically corrupted, raising no exception. This measures the wrapped tools; the agent-generated
 140 code, which need not rely on any single parser, is the subject of §4.2.

Arm	CF/grammar (<i>n</i> =4)	mildly-CS (<i>n</i> =26)	Total (<i>n</i> =30)
NONE (no harness)	75	50	53
BARE	100	54	60
DOCS	100	54	60
FULL	100	58	63
COMPLEXITY	100	58	63

Table 1: Realized extraction success (%) by grammatical (Chomsky/Joshi) class and arm, over an identical 30-task set (single-run, Claude Opus 4.8). The regular rung is omitted (no example in the corpus, §4.1); the single context-sensitive task, three GAMESS tasks, and one oversized GPAW output are excluded (the last two for runtime reasons; Appendix A). Success rises only modestly with added support; the class gradient (CF vs. mildly-CS) is the larger effect.

Re-cut by grammatical class (Table 4), best-of-four recovery is 100% on the context-free formats and 43% on the mildly context-sensitive logs. The gradient tracks grammar-availability rather than class as such: the context-free formats, though grammatically more complex, are recovered whenever a supplied grammar exists, whereas the plainer free-form logs without a canonical grammar are not. A supplied grammar, not grammatical simplicity, appears to drive extraction reliability.

4.2 Main: realized capability and failure modes by arm

We compare all five arms on an identical set of 30 tasks (Table 1 for success, Figure 1 for failure modes; the single context-sensitive task is held out). These are observations from a first sweep, not confirmed effects: single runs per cell and one model preclude significance claims, and per-run results and chain-of-thought transcripts appear in the appendices. Three results stand out; success does not rise monotonically with harness support.

(O1) Exposing tools barely moves success but adds cost and silent failures. From NONE to the plain tool arms, total success rises only modestly (53% to 60–63%), while average tool calls *rise* (5.4 to 7.4) and two failure modes appear that the from-scratch baseline never exhibits: wrong-parser calls (0 to ~21) and, more troubling, silent-empty outputs (0 to ~11), where the agent accepts an empty parse and emits a hollow result with no error. Per-parser documentation (DOCS, FULL) does not remove these.

(O2) The complexity annotation is what shifts the failure profile. COMPLEXITY attains the same success as FULL (63%) but nearly eliminates the interface failures—wrong-parser 21 → 2, silent-empty 11 → 1—and returns effort to near-baseline (5.9 tool calls). Telling the agent where the source sits on the hierarchy and where each tool’s power tops out, rather than merely which formats each tool nominally supports, appears to be the operative ingredient.

(O3) Reasoning failures are untouched. The gave-up rate (missing core fields) is essentially flat across every arm (11–14), including NONE, and no harness variant we tested changed it.

5 Discussion and outlook

At the tool level, these first sweeps read as feedback on the harness rather than the agent: the mismatches above are a failure of tooling, not of the agent, which merely acts on the unreliable or overstated information it is given. These tests also expose two structural shortcomings in the MCP server itself. First, its capability metadata overstates which formats each parser handles, so an agent that trusts it calls a parser that returns nothing. Second, its Pydantic output contract is permissive enough that a hollow parse validates as a well-formed object, leaving the emptiness silent. Both are addressable—through tighter capability descriptions and stricter output constraints (required fields, non-empty collections, cross-field consistency)—and are slated for correction before the next round of tests. The semantic gap runs deeper: confirming that a non-empty, internally consistent result holds the *correct* values exceeds what a permissive type contract can express, and may demand a more expressive discipline than Python affords—dependent types, for instance—or validation outside the type system altogether.

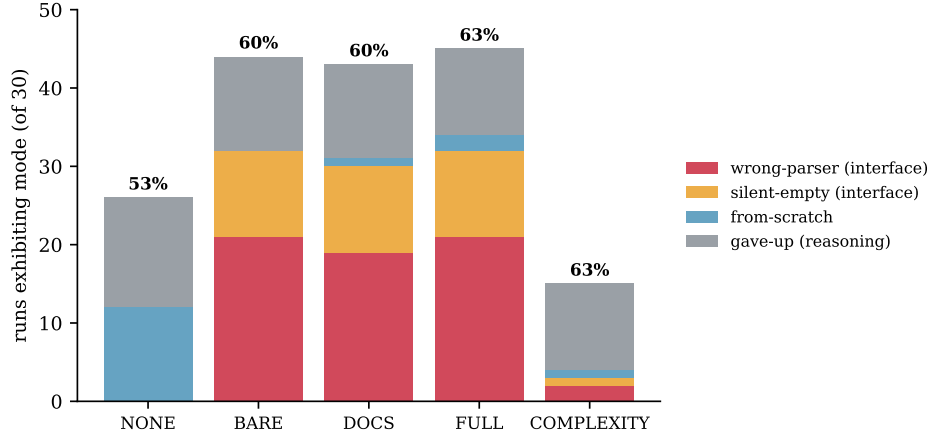


Figure 1: Failure-mode counts by arm over the 30-task set (overall success rate shown above each bar). *Wrong-parser* (called a parser that cannot handle the format) and *silent-empty* (emitted output despite an empty parse) are interface failures; *from-scratch* hand-rolls a parser; *gave-up* (missing core fields) is the reasoning failure. Exposing tools introduces the interface failures; only COMPLEXITY removes them, while gave-up and overall success stay roughly flat. The near-zero unit-error and retry-loop modes (≤ 1 each) are omitted.

178 The larger effect is the opposite. What most improved the agent’s behaviour was not more tooling
 179 but *knowledge*: a short statement of where the source sits in the language hierarchy, and where each
 180 tool’s power ends, collapsed the interface failures at no extra cost. Why this context helps, and
 181 how the agent uses it, is best read from the chain-of-thought; extending coverage to the multi-file
 182 (context-sensitive) case is the next step. The Turing-complete extreme, by contrast, amounts to code
 183 analysis and is already well studied elsewhere—a known bound rather than a target for this line.

184 Limitations

185 This is a first, small study and its numbers are indicative, not conclusive. The agent sweep uses
 186 a single model (Claude Opus 4.8), a single target schema, and one run per cell, so we make no
 187 significance claims and cannot separate model-specific behaviour from general effects. The cor-
 188 pus is a single database snapshot (51 entries, 16 codes) heavily skewed toward the mildly-context-
 189 sensitive band, and the COMPLEXITY arm bundles two cues—the source’s grammatical class and
 190 each tool’s power ceiling—that the design does not separate. Further methodological caveats—the
 191 union-of-parsers reference in place of independent ground truth, the heuristic failure detectors, and
 192 the infrastructure exclusions—are detailed in the appendix.

References

- Anthropic. Introducing the Model Context Protocol. <https://www.anthropic.com/news/model-context-protocol>, 2024. Accessed 2026-08-29.
- Noam Chomsky. Three models for the description of language. *IRE Transactions on Information Theory*, 2(3): 113–124, 1956.
- Noam Chomsky. *Aspects of the Theory of Syntax*. MIT Press, 1965.
- Grégoire Delétang, Anian Ruoss, Jordi Grau-Moya, Tim Genewein, Li Kevin Wenliang, Elliot Catt, Chris Cundy, Marcus Hutter, Shane Legg, Joel Veness, and Pedro A. Ortega. Neural networks and the Chomsky hierarchy. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2207.02098>. arXiv:2207.02098.
- Dong et al. Evaluating the formal reasoning capabilities of large language models through the Chomsky hierarchy. *arXiv preprint*, 2026. URL <https://arxiv.org/abs/2604.02709>. arXiv:2604.02709.
- Claudia Draxl and Matthias Scheffler. The NOMAD laboratory: from data sharing to artificial intelligence. *Journal of Physics: Materials*, 2(3):036001, 2019. doi: 10.1088/2515-7639/ab13bb.
- Nouha Dziri et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.18654.
- Luca M. Ghiringhelli, Carsten Baldauf, Tristan Bereau, Sandor Brockhauser, Christian Carbogno, Javad Chamanara, et al. Shared metadata for data-centric materials science. *Scientific Data*, 10:626, 2023. doi: 10.1038/s41597-023-02501-8. arXiv:2205.14774.
- Aravind K. Joshi. Tree adjoining grammars: How much context-sensitivity is required to provide reasonable structural descriptions? In *Natural Language Parsing*. Cambridge University Press, 1985.
- William Merrill and Ashish Sabharwal. The expressive power of transformers with chain of thought. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2310.07923>. arXiv:2310.07923.
- Jorge Pérez, Pablo Barceló, and Javier Marinkovic. Attention is Turing-complete. *Journal of Machine Learning Research*, 22(75):1–35, 2021.
- Jackson Petty, Michael Y. Hu, Wang, Shauli Ravfogel, William Merrill, and Tal Linzen. RELIC: Evaluating complex reasoning via the recognition of languages in-context. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2506.05205>. arXiv:2506.05205.
- Hiroyuki Seki, Takashi Matsumura, Mamoru Fujii, and Tadao Kasami. On multiple context-free grammars. *Theoretical Computer Science*, 88(2):191–229, 1991.

A Experimental detail

The harness and experiment code, anonymized for review, are available at <https://anonymous.4open.science/r/parse-patrol-C611/>.

Each run gives one coding agent (Claude Opus 4.8, via the Claude Agent SDK) a single source file, the fixed target schema, and code execution, capped at 30 turns and a per-run spend budget; only the arm’s system-prompt support varies, as described in the Method. Success is field-level agreement with a reference built from the union of all succeeding parsers on that file rather than an independent ground truth, and the failure-mode labels come from conservative, transcript-grounded detectors; both are therefore approximate, and a run is counted toward a mode when it *exhibits* that mode (a successful run may still have made, e.g., a wrong-parser call en route). The grammatical-class assignment is a documented ordinal, partly confounded with program at the small per-class sample sizes.

The 16 simulation codes spanned by the corpus are ABINIT, CASTEP, CP2K, Crystal, exciting, FHI-aims, GAMESS, Gaussian, GPAW, LAMMPS, NWChem, Octopus, ONETEP, ORCA, VASP, and WIEN2k. Three GAMESS tasks, whose parsed model exceeds the agent runtime’s message-size limit, and one ~3.5 GB GPAW output, which hangs the run, are excluded from the agent sweep—infrastructure constraints, not parsing outcomes.

B Per-task outcomes

Table 2 gives the success of every arm on each of the 30 matched tasks. The pattern behind the aggregate numbers is visible here: on solvable tasks the tool arms match NONE or improve on it, while on the hard mildly-context-sensitive logs (ABINIT, CASTEP, CP2K, LAMMPS, Octopus, exciting) all arms fail—but, as Figure 1 shows, the tool arms reach that failure by calling a mismatched parser and emitting an empty-but-accepted result, whereas COMPLEXITY more often fails cleanly.

Program	Class	NONE	BARE	DOCS	FULL	COMPLEXITY
VASP	CF	✓	✓	✓	✓	✓
VASP	CF	✓	✓	✓	✓	✓
VASP	CF	—	✓	✓	✓	✓
VASP	CF	✓	✓	✓	✓	✓
ABINIT	mCS	—	—	—	—	—
ABINIT	mCS	—	—	—	—	—
CASTEP	mCS	—	—	—	—	—
CASTEP	mCS	—	—	—	—	—
CP2K	mCS	—	—	—	—	—
Crystal	mCS	—	—	—	—	—
FHI-aims	mCS	✓	✓	✓	✓	✓
FHI-aims	mCS	✓	✓	✓	✓	✓
FHI-aims	mCS	✓	✓	✓	✓	✓
FHI-aims	mCS	✓	✓	✓	✓	✓
GPAW	mCS	✓	✓	✓	✓	✓
GPAW	mCS	✓	✓	✓	✓	✓
Gaussian	mCS	—	✓	✓	✓	✓
Gaussian	mCS	✓	✓	✓	✓	✓
Gaussian	mCS	✓	✓	✓	✓	✓
Gaussian	mCS	✓	✓	✓	✓	✓
LAMMPS	mCS	—	—	—	—	—
NWChem	mCS	—	—	—	—	—
NWChem	mCS	✓	✓	✓	✓	✓
ONETEP	mCS	—	—	—	—	—
ONETEP	mCS	—	—	—	✓	✓
ORCA	mCS	✓	✓	✓	✓	✓
ORCA	mCS	✓	✓	✓	✓	✓
ORCA	mCS	✓	✓	✓	✓	✓
Octopus	mCS	—	—	—	—	—
exciting	mCS	—	—	—	—	—

Table 2: Per-task success (✓) by arm over the 30 matched tasks (CF: context-free with supplied grammar; mCS: mildly context-sensitive). Class column uses short labels.

247 C Representative chain-of-thought

248 The contrast is visible in the transcripts. On a CASTEP log (hard, no supplied grammar), the BARE
 249 agent follows the tool affordance—“CASTEP is best handled by [two parsers]; let me parse with
 250 both”—then “both parsers failed,” and proceeds to emit a partly-empty result. The COMPLEXITY
 251 agent, told the source is a free-form log outranking every available parser, skips the tools and reads
 252 the file directly (“this is a CASTEP single-point energy calculation; let me compute the conversions
 253 and write the output”). The difference is not capability—both can read a file—but knowing in
 254 advance that the tools will not fit.

255 D Preliminary component results

256 Tables 3 and 4 give the parser-benchmark measurements summarized in §4.1: per-parser structure-
 257 recovery reliability over 1,104 files, and the same recovery re-cut by grammatical class.

Parser	All files	Primary output
A (atomistic)	3.9%	23.5%
B (quantum-chem)	15.4%	21.6%
C (regex)	2.4%	7.8%
D (I/O lib)	10.5%	5.9%
Best-of-four (any)	20.5%	45.1%

Table 3: Structure-recovery reliability (preliminary component result).

Source class (primary output)	Best-of-four recovery
context-free, grammar supplied (e.g. XML)	100%
mildly context-sensitive (free-form log, count-agreement)	43%

Table 4: Best-of-four recovery by grammatical class, over the two well-populated rungs (the regular and context-sensitive extremes are under-represented in the corpus and omitted). The context-free “100%” reflects a *supplied* grammar, not grammatical simplicity (preliminary).